

# 4 – Versions distribuées

Hadrien Grasland

IJCLab – Orsay

24/11/2023

# Introduction

- **Jusqu'ici, nous n'avons utilisé qu'un seul dépôt**
  - Tous les commits y étaient centralisés
  - Git avait accès à l'ensemble de ceux-ci
  - Toute divergence d'historique était volontaire
- **Nous allons maintenant combiner plusieurs dépôts**
  - L'outil n'aura plus une vision omnisciente
  - Prérequis pour une utilisation en réseau

# Dépôts distants

- **Tout dépôt git peut être connecté à d'autres dépôts**
  - On parle de dépôts distants ou *remotes*
  - Différents protocoles : système de fichier, HTTPS, SSH…
- **La connexion permet deux nouvelles opérations :**
  - Télécharger *commits* & références (branches, étiquettes)
  - Soumettre des changements au dépôt distant
- **Tout échange doit être demandé explicitement**

# Problème des collisions

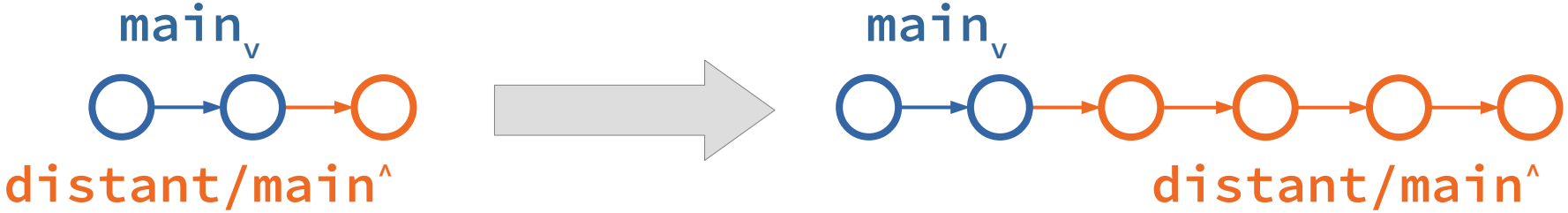
- **En distribué, il faut gérer les collisions**
  - Deux objets de même nom, mais de contenu différent
- **Une collision de commits est peu probable**
  - Leur identifiant est fonction du contenu (*hash*)
- **On peut en avoir entre branches, étiquettes…**
  - Leur identifiant est choisi par l'utilisateur
  - Des références locales et distantes peuvent diverger

# Branches et étiquettes distantes

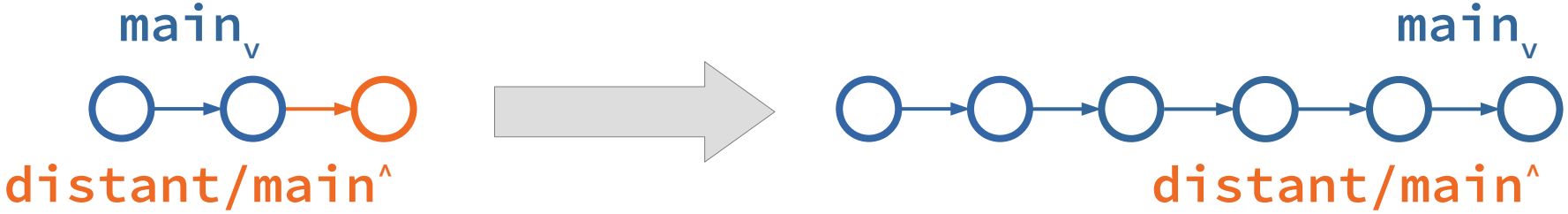
- **Git nomme les « objets distants » <remote>/<nom>**
  - Ce sont en fait des copies locales d'objets distants
  - « git fetch » met à jour ces copies locales
  - Ce nommage diminue les risques de collisions
- **On peut synchroniser branches locales et distantes**
  - Avec les outils vus précédemment : merge, rebase…
  - …ou avec des commandes spécialisées : *push*, *pull*

# Récupérer des changements

- *Fetch* : Mise à jour des références distantes

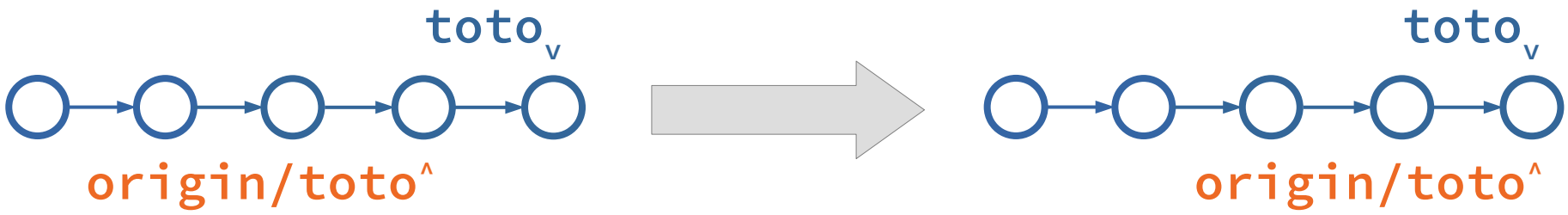



- *Pull* : *Fetch* puis fusion d'une branche



# Soumettre des changements

- *Push :* Créer ou avancer une branche distante




- **Attention, la relation « remote » est asymétrique :**
  - A peut avoir B en remote sans que l'inverse soit vrai
  - Il n'existe pas d'opération inverse de *fetch*
  - *Pull* tolère les divergences, alors que *push* les rejette

# Un point de syntaxe

- **Quand on télécharge des branches distantes…**
  - git maintient une copie locale « <remote>/<branche> »
  - Représente un objet distant donc en lecture seule
- **Quand on interagit avec une branche distante…**
  - La syntaxe est « <commande> <remote> <branche> »
  - Peut modifier des branches, locales ou sur <remote>
- **Attention : syntaxe proche, effets différents !**

# Démo : Dépôts distants dans gitg

# TP : Dépôts distribués